

Bridging Curriculum and Industry: Identifying Applied Computer Science Concepts in a Software Internship and Designing a Containerization, Kubernetes, and Scalability Laboratory

Independent Study Report

Student: Ethan Bogart

Advisor: Ted Lehr

Course Number: 4395

Date of Submission: July 14, 2026

Institution: Texas State University, Department of Computer Science

Abstract

This independent study examines the relationship between the undergraduate computer science curriculum at Texas State University and the technical work performed during a professional software engineering internship. The study was conducted through independent online research combined with mentorship from a cloud architect at the internship site. Its original goal was to identify which curriculum concepts are applied in a real professional environment and which important skills are used in industry but are not covered in coursework. The investigation found strong alignment in foundational areas such as version control, Agile methodology, object-oriented programming, API design, and software engineering practice, but a significant gap in the areas of containerization, multi-service application composition, horizontal scalability, and container orchestration with Kubernetes. As these gaps became clear, the focus of the study shifted from cataloging missing concepts to producing a concrete deliverable: a three-phase, hands-on laboratory assignment that teaches Docker image creation, multi-container composition with a database and background worker, and cloud deployment on Azure Kubernetes Service (AKS) with automatic scaling. The resulting assignment is designed to be adopted by faculty so that future students gain practical, industry-relevant experience with the same technologies encountered in the internship.

1. Motivation and Introduction

University computer science programs must balance timeless fundamentals against the fast-moving tools and platforms used in industry. Students frequently ask a practical question: how much of what I learn in the classroom will I actually use on the job, and what will I have to learn on my own? During my software engineering internship I experienced both sides of this question directly. Much of my coursework transferred immediately and gave me a strong foundation, yet a large and central portion of my day-to-day work relied on technologies I had never encountered in any course.

This study was motivated by that question. The goal was first to identify which concepts from the Texas State computer science curriculum are used in a real professional work environment, and second to identify the important skills and knowledge used in the internship that were not covered in my coursework. By comparing course concepts with the work I performed at my internship, the study highlights which parts of the curriculum have direct practical value and which areas may benefit from additional real-world preparation.

As the study progressed, it became clear that simply naming the missing topics was not the most valuable outcome. The most useful contribution would be to close the gap directly. The study therefore evolved into the design of a practical assignment that Professor Ted Lehr can work off and use to teach the missing skills, giving students exposure to containerization, Kubernetes, and scalability before they reach the workplace.

2. Approach and Methodology

The study combined two complementary methods: independent online research and direct mentorship from a staff member at my internship, specifically the organization's cloud architect. The mentorship provided real-world context and guidance on professional tooling and best practices, while the independent research provided the depth needed to understand the underlying technologies and to build a teachable artifact.

The work followed a sequence of milestones:

- Establish the scope and method of the study.
- Document internship responsibilities, tools, and applied skills.
- Identify course concepts that are directly used in the internship.
- Identify gaps between coursework and internship expectations.
- Design a sample assignment based on the most significant identified gap.
- Prepare a final report and presentation summarizing the findings.

To keep the comparison grounded, internship tasks were mapped against specific curriculum concepts. Where a concept from a course was used on the job, it was recorded as an alignment. Where a task required knowledge with no clear classroom counterpart, it was recorded as a gap. The largest and most consequential gap identified was containerization and orchestration, which became the subject of the designed assignment.

3. Results and Outcome

3.1 Curriculum Concepts Applied in the Internship

Several core areas of the Texas State curriculum transferred directly to professional work and formed the everyday foundation of my contributions:

Concept / Skill (taught at Texas State)	How it was applied in the internship
Version control with Git	Branching, commits, pull requests, and code review as part of the daily development workflow.
Agile methodology and Jira	Sprint planning, task tracking, stand-ups, and issue management in an Agile team.
Object-oriented programming (OOP)	Structuring application code into maintainable classes and modules.
APIs and web services	Building and consuming RESTful HTTP endpoints to move data between services.
Programming courses (data structures, logic)	Implementing application logic, handling data, and reasoning about correctness.
Software engineering practices	Requirements, design, testing, documentation, and collaborative development.

These findings confirm that the curriculum provides genuine, practical value. The foundational skills were not merely academic; they were prerequisites for being productive on a professional team.

3.2 Gaps Between Coursework and Internship Expectations

A substantial portion of the internship work depended on technologies that were not part of my coursework. These were learned on the job with the guidance of the cloud architect:

- Containerization with Docker — packaging an application and its dependencies into a portable image.
- Multi-container composition — running and connecting several services (web API, background worker, database) with Docker Compose.
- Persistent state and data volumes — keeping database data alive independently of container lifecycles.
- Horizontal scalability — running multiple copies of a service to handle increased load.
- Container orchestration with Kubernetes — deploying, networking, and managing containers on a cluster.
- Cloud deployment and managed clusters

These topics are central to modern software delivery, yet they received no coverage in my coursework. Because so much professional work now happens inside containers and orchestrated clusters, this gap represents a meaningful opportunity for the curriculum.

3.3 Primary Deliverable: A Three-Phase Containerization and Kubernetes Lab

The main outcome of this study is a hands-on laboratory assignment that teaches the identified gap directly. Rather than only reporting that these skills are missing, the study produced a ready-to-use assignment that a professor can assign to students. The lab is organized into three progressive phases, each building on the previous one:

- Phase 1 - Docker Fundamentals: students build and run a container from a Dockerfile, learning images, layers, ports, and the container lifecycle.
- Phase 2 - Multi-Container Application: students compose a realistic system with three services - an Express web API, a background worker, and a MongoDB database - using Docker Compose, environment variables, a shared network, and a persistent volume. The API accepts a prompt, stores it as a job, the worker processes the job (calling an AI chat API or a demo response), and the result is written back to MongoDB.
- Phase 3 - Kubernetes and Scalability on Azure: students push their images to Azure Container Registry and deploy the same application to Azure Kubernetes Service. They learn Kubernetes objects (Deployments, StatefulSets, Services, PersistentVolumeClaims), expose the app through a public load balancer, and configure automatic scaling. The phase demonstrates two scaling strategies matched to the workload: a CPU-based Horizontal Pod Autoscaler for the web API and event-driven, queue-based scaling with KEDA for the background worker.

A key teaching point embedded in Phase 3 is choosing the right scaling signal for each component. The web API is CPU-bound under request load, so a CPU-based autoscaler is appropriate. The worker, by contrast, spends most of its time waiting on an external call and therefore uses very little CPU; scaling it on CPU would fail. Instead it is scaled on the depth of the job queue in the database using KEDA. This distinction mirrors a real design decision made during the internship and is exactly the kind of practical judgment that is difficult to teach without a working system to experiment on.

The assignment includes a complete, line-by-line explanation of every configuration file so that students understand not just how to run the commands but why each piece exists. It also includes a single workload-generation script that drives both autoscalers, lets students watch new pods being created and then removed as load rises and falls, and automatically pauses the cloud cluster afterward to conserve credits.

4. Conclusion and Future Work

This study confirmed that the Texas State computer science curriculum delivers strong, directly applicable foundations - version control, Agile practice, object-oriented programming, APIs, and software engineering - that made me productive during my internship. It also identified a clear and important gap: containerization, multi-service composition, scalability, and Kubernetes orchestration are central to modern industry work but were absent from my coursework. Recognizing that the most valuable response to a gap is to close it, the study produced a

practical, three-phase lab assignment that teaches these skills using the same tools and design decisions I encountered professionally.

Several directions could extend this work:

- Pilot the lab in a course and collect student feedback to refine the difficulty and pacing of each phase.
- Add automated grading or verification scripts so instructors can check student deployments quickly.
- Introduce a CI/CD pipeline phase that builds and deploys the images automatically, another common industry practice.
- Extend the observability portion with metrics dashboards and logging so students can see system behavior more clearly.
- Generalize the study by surveying additional interns to identify other recurring curriculum gaps.

In its final form, this independent study is both an analysis and a contribution: it documents where classroom learning met professional reality, and it delivers a concrete teaching artifact intended to give future students the practical, industry-relevant experience that the analysis showed was missing.

References

- [1] Docker, Inc. "Docker Documentation." <https://docs.docker.com/>
- [2] Docker, Inc. "Docker Compose Documentation." <https://docs.docker.com/compose/>
- [3] The Kubernetes Authors. "Kubernetes Documentation." <https://kubernetes.io/docs/>
- [4] The Kubernetes Authors. "Horizontal Pod Autoscaling." <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- [5] KEDA. "Kubernetes Event-driven Autoscaling - Documentation." <https://keda.sh/docs/>
- [6] KEDA. "MongoDB Scaler." <https://keda.sh/docs/latest/scalers/mongodb/>
- [7] Microsoft. "Azure Kubernetes Service (AKS) Documentation." <https://learn.microsoft.com/azure/aks/>
- [8] Microsoft. "Azure Container Registry Documentation." <https://learn.microsoft.com/azure/container-registry/>
- [9] MongoDB, Inc. "MongoDB Manual." <https://www.mongodb.com/docs/manual/>
- [10] Ty Acker cloud engineer/solution architect.